

C++ Programming

Day 14: Integration and Program Structure

Program Structure, Class + vector, find_if, File Save/Load, Exception Handling

Contents

1	학습 목표	2
2	Day 14 개요	2
3	Program Structure	2
3.1	메뉴 반복 구조	2
3.2	최종 코드	3
3.3	실습	4
4	Class와 vector 통합	5
4.1	객체 컨테이너	5
4.2	최종 코드	5
4.3	실습	6
5	Search: find_if와 Lambda	8
5.1	검색 구조	8
5.2	최종 코드	8
5.3	실습	9
6	File Save: ofstream	10
6.1	객체 데이터 저장	11
6.2	최종 코드	11
6.3	실습	12
7	File Load: ifstream	13
7.1	파일에서 객체 복원	13
7.2	최종 코드	13
7.3	실습	14
8	Exception Handling 통합	16
8.1	파일 오류와 runtime_error	16
8.2	최종 코드	16
8.3	실습	17
9	개념 연결	18
10	종합 정리	19
11	실습 파일 구성	20

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 메뉴 기반 프로그램의 전체 흐름을 함수로 분리하기
- 클래스 객체를 `vector`에 저장하고 여러 객체를 관리하기
- 참조 전달과 `const` 참조를 상황에 맞게 사용하기
- `find_if`와 `lambda`를 이용해 객체 컨테이너에서 원하는 데이터를 검색하기
- `ofstream`으로 객체 데이터를 파일에 저장하기
- `ifstream`으로 파일 데이터를 읽어 객체 컨테이너로 복원하기
- `try`, `throw`, `catch`를 파일 처리와 연결하기
- Day 1-13에서 배운 문법을 하나의 프로그램 구조 안에서 연결하기

2. Day 14 개요

Day 14는 새로운 문법을 대량으로 학습하는 날이 아니라, Day 1-13에서 배운 핵심 개념을 실제 프로그램 구조 안에서 연결하는 통합 복습이다. 개념 예제에서는 `Book`을 사용하고, 각 실습에서는 같은 구조를 `Movie`에 적용한다. 이를 통해 예제 코드를 그대로 복사하는 것이 아니라 동일한 개념을 다른 객체에 적용하는 연습을 한다.

주제	핵심 역할
Program Structure	메뉴와 기능을 함수로 분리
Class + <code>vector</code>	여러 객체를 하나의 컨테이너에서 관리
<code>find_if</code> + <code>lambda</code>	조건에 맞는 객체 검색
File Save	객체 데이터를 파일에 저장
File Load	파일 데이터를 객체로 복원
Exception Handling	오류 발생과 처리 흐름 분리

핵심 포인트

Day 14의 핵심은 개별 문법을 다시 외우는 것이 아니라, 클래스, STL, 함수, 파일 입출력, 예외 처리가 하나의 프로그램 안에서 어떤 순서로 연결되는지를 이해하는 것이다.

3. Program Structure

작은 예제에서는 모든 코드를 `main()`에 작성할 수 있지만, 프로그램이 커질수록 메뉴 출력, 데이터 추가, 검색, 저장과 같은 기능을 함수로 분리하는 것이 중요하다. `main()`은 세부 작업을 직접 수행하기보다 프로그램의 전체 흐름을 관리하도록 구성할 수 있다.

3.1. 메뉴 반복 구조

메뉴 기반 프로그램에서는 보통 반복문 안에서 메뉴를 출력하고 사용자의 선택을 입력받는다.

```
1 while (true)
2 {
```

```
3     showMenu();
4     cin >> choice;
5
6     if (choice == 0)
7     {
8         break;
9     }
10 }
```

showMenu()와 입력이 반복문 안에 있어야 매 반복마다 새로운 선택을 받을 수 있다.

3.2. 최종 코드

```
1 #include <iostream>
2
3 using namespace std;
4
5 void showMenu()
6 {
7     cout << "==== Book Manager =====> endl;
8     cout << "1. Add Book" << endl;
9     cout << "2. Show Books" << endl;
10    cout << "3. Find Book" << endl;
11    cout << "0. Exit" << endl;
12    cout << "Select: ";
13 }
14
15 int main()
16 {
17     int choice;
18
19     while (true)
20     {
21         showMenu();
22         cin >> choice;
23
24         if (choice == 1)
25         {
26             cout << "Add Book selected." << endl;
27         }
28         else if (choice == 2)
29         {
30             cout << "Show Books selected." << endl;
31         }
32         else if (choice == 3)
33         {
34             cout << "Find Book selected." << endl;
35         }
36         else if (choice == 0)
37         {
38             cout << "Program finished." << endl;
39             break;
40         }
41         else
42         {
43             cout << "Invalid choice." << endl;
44         }
45
46         cout << endl;
```

```
47     }
48
49     return 0;
50 }
```

3.3. 실습

실습

Movie Manager 메뉴를 만든다. showMenu() 함수를 사용하고 while (true) 안에서 메뉴 출력과 입력을 반복한다. 1, 2, 3은 각각 Add Movie, Show Movies, Find Movie를 선택하고, 0은 프로그램을 종료한다.

```
1  #include <iostream>
2
3  using namespace std;
4
5  void showMenu()
6  {
7      cout << "==== Movie Manager =====" << endl;
8      cout << "1. Add Movie" << endl;
9      cout << "2. Show Movies" << endl;
10     cout << "3. Find Movie" << endl;
11     cout << "0. Exit" << endl;
12     cout << "Select: ";
13 }
14
15 int main()
16 {
17     int choice;
18
19     while (true)
20     {
21         showMenu();
22         cin >> choice;
23
24         if (choice == 1)
25         {
26             cout << "Add Movie selected" << endl;
27         }
28         else if (choice == 2)
29         {
30             cout << "Show Movies selected" << endl;
31         }
32         else if (choice == 3)
33         {
34             cout << "Find Movie selected" << endl;
35         }
36         else if (choice == 0)
37         {
38             cout << "Program finished" << endl;
39             break;
40         }
41         else
42         {
43             cout << "Invalid choice." << endl;
44         }
45     }
```

```

46     cout << endl;
47 }
48
49 return 0;
50 }

```

4. Class와 vector 통합

클래스는 하나의 객체 구조를 정의하고, `vector`는 같은 타입의 여러 데이터를 관리한다. 따라서 `vector<Book>`을 사용하면 여러 `Book` 객체를 하나의 컨테이너에서 관리할 수 있다.

4.1. 객체 컨테이너

다음 선언은 정수가 아니라 `Book` 객체를 저장하는 `vector`를 만든다.

```
1 vector<Book> books;
```

객체는 `push_back()`으로 추가할 수 있다.

```
1 books.push_back(Book("C++", 30000));
```

함수가 원본 `vector`를 수정해야 한다면 참조 전달을 사용한다.

```
1 void addBook(vector<Book>& books)
```

반대로 출력처럼 읽기만 하는 함수는 `const` 참조를 사용할 수 있다.

```
1 void showBooks(const vector<Book>& books)
```

4.2. 최종 코드

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 using namespace std;
6
7 class Book
8 {
9 public:
10     string title;
11     int price;
12
13     Book(string title, int price)
14     {
15         this->title = title;
16         this->price = price;
17     }
18
19     void showInfo() const
20     {
21         cout << "Title: " << title << endl;
22         cout << "Price: " << price << endl;
23     }
24 };
25
26 void addBook(vector<Book>& books)
27 {

```

```

28     string title;
29     int price;
30
31     cout << "Title: ";
32     cin >> title;
33
34     cout << "Price: ";
35     cin >> price;
36
37     books.push_back(Book(title, price));
38
39     cout << "Book added." << endl;
40 }
41
42 void showBooks(const vector<Book>& books)
43 {
44     if (books.empty())
45     {
46         cout << "No books." << endl;
47         return;
48     }
49
50     for (const Book& book : books)
51     {
52         book.showInfo();
53         cout << endl;
54     }
55 }
56
57 int main()
58 {
59     vector<Book> books;
60
61     addBook(books);
62     addBook(books);
63
64     cout << endl;
65
66     showBooks(books);
67
68     return 0;
69 }

```

4.3. 실습

실습

Movie 클래스를 만들고 `vector<Movie>`에 여러 영화를 저장한다. `addMovie(vector<Movie>& movies)`로 원본 `vector`에 영화를 추가하고, `showMovies(const vector<Movie>& movies)`로 모든 영화를 출력한다.

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 using namespace std;
6
7 class Movie

```

```
8 {
9 public:
10     string title;
11     int rating;
12
13     Movie(string title, int rating)
14     {
15         this->title = title;
16         this->rating = rating;
17     }
18
19     void showInfo() const
20     {
21         cout << "Title: " << title << endl;
22         cout << "Rating: " << rating << endl;
23     }
24 };
25
26 void addMovie(vector<Movie>& movies)
27 {
28     string title;
29     int rating;
30
31     cout << "Title: ";
32     cin >> title;
33
34     cout << "Rating: ";
35     cin >> rating;
36
37     movies.push_back(Movie(title, rating));
38 }
39
40 void showMovies(const vector<Movie>& movies)
41 {
42     if (movies.empty())
43     {
44         cout << "No movies" << endl;
45         return;
46     }
47
48     for (const Movie& movie : movies)
49     {
50         movie.showInfo();
51         cout << endl;
52     }
53 }
54
55 int main()
56 {
57     vector<Movie> movies;
58
59     addMovie(movies);
60     addMovie(movies);
61
62     cout << endl;
63
64     showMovies(movies);
65
66     return 0;
67 }
```


5. Search: find_if와 Lambda

find_if는 컨테이너에서 특정 조건을 만족하는 첫 번째 원소를 찾는 STL 알고리즘이다. 객체의 멤버 값을 기준으로 검색할 때 lambda를 조건 함수로 사용할 수 있다.

5.1. 검색 구조

```
1 auto it = find_if(
2     books.begin(),
3     books.end(),
4     [&target](const Book& book)
5     {
6         return book.title == target;
7     }
8 );
```

lambda는 각 Book을 검사하고 제목이 target과 같으면 true를 반환한다. 검색에 성공하면 iterator가 해당 객체를 가리키고, 실패하면 books.end()가 반환된다.

```
1 if (it != books.end())
2 {
3     it->showInfo();
4 }
```

5.2. 최종 코드

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5
6 using namespace std;
7
8 class Book
9 {
10 public:
11     string title;
12     int price;
13
14     Book(string title, int price)
15     {
16         this->title = title;
17         this->price = price;
18     }
19
20     void showInfo() const
21     {
22         cout << "Title: " << title << endl;
23         cout << "Price: " << price << endl;
24     }
25 };
26
27 void findBook(const vector<Book>& books)
28 {
29     string target;
```

```

30
31     cout << "Title to find: ";
32     cin >> target;
33
34     auto it = find_if(
35         books.begin(),
36         books.end(),
37         [&target](const Book& book)
38         {
39             return book.title == target;
40         }
41     );
42
43     if (it != books.end())
44     {
45         cout << "Book found." << endl;
46         it->showInfo();
47     }
48     else
49     {
50         cout << "Book not found." << endl;
51     }
52 }
53
54 int main()
55 {
56     vector<Book> books;
57
58     books.push_back(Book("C++", 30000));
59     books.push_back(Book("Python", 25000));
60     books.push_back(Book("Physics", 40000));
61
62     findBook(books);
63
64     return 0;
65 }

```

5.3. 실습

실습

vector<Movie>에서 사용자가 입력한 제목과 일치하는 영화를 찾는다. find_if와 lambda를 사용하고, 검색에 성공하면 영화 정보를 출력한다.

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <algorithm>
5
6  using namespace std;
7
8  class Movie
9  {
10 public:
11     string title;
12     int rating;
13
14     Movie(string title, int rating)

```

```
15 {
16     this->title = title;
17     this->rating = rating;
18 }
19
20 void showInfo() const
21 {
22     cout << "Title: " << title << endl;
23     cout << "Rating: " << rating << endl;
24 }
25 };
26
27 void findMovie(const vector<Movie>& movies)
28 {
29     string target;
30
31     cout << "Enter the title of movie you want to find: ";
32     cin >> target;
33
34     auto it = find_if(
35         movies.begin(),
36         movies.end(),
37         [&target](const Movie& movie)
38         {
39             return movie.title == target;
40         }
41     );
42
43     if (it != movies.end())
44     {
45         cout << "Movie found" << endl;
46         it->showInfo();
47     }
48     else
49     {
50         cout << "Movie not found" << endl;
51     }
52 }
53
54 int main()
55 {
56     vector<Movie> movies;
57
58     movies.push_back(Movie("Interstellar", 9));
59     movies.push_back(Movie("Inception", 8));
60     movies.push_back(Movie("Tenet", 7));
61
62     findMovie(movies);
63
64     return 0;
65 }
```

6. File Save: ofstream

프로그램 내부의 객체 데이터는 프로그램이 종료되면 사라진다. ofstream을 사용하면 vector<Book>의 데이터를 외부 파일에 저장할 수 있다.

6.1. 객체 데이터 저장

```

1 ofstream file("books.txt");
2
3 for (const Book& book : books)
4 {
5     file << book.title << " " << book.price << endl;
6 }

```

파일에 저장만 하므로 vector와 각 객체를 수정할 필요가 없다. 따라서 `const` 참조를 사용한다.

6.2. 최종 코드

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <fstream>
5
6 using namespace std;
7
8 class Book
9 {
10 public:
11     string title;
12     int price;
13
14     Book(string title, int price)
15     {
16         this->title = title;
17         this->price = price;
18     }
19 };
20
21 void saveBooks(const vector<Book>& books)
22 {
23     ofstream file("books.txt");
24
25     if (!file.is_open())
26     {
27         cout << "Failed to open file." << endl;
28         return;
29     }
30
31     for (const Book& book : books)
32     {
33         file << book.title << " " << book.price << endl;
34     }
35
36     file.close();
37
38     cout << "Books saved." << endl;
39 }
40
41 int main()
42 {
43     vector<Book> books;
44
45     books.push_back(Book("C++", 30000));
46     books.push_back(Book("Python", 25000));

```

```

47     books.push_back(Book("Physics", 40000));
48
49     saveBooks(books);
50
51     return 0;
52 }

```

파일에는 다음 내용이 저장된다.

```

C++ 30000
Python 25000
Physics 40000

```

6.3. 실습

실습

vector<Movie>에 저장된 영화 정보를 movies.txt에 저장한다. 각 줄은 title rating 형식으로 저장하고, 파일 열기에 실패하면 오류 메시지를 출력한다.

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <fstream>
5
6  using namespace std;
7
8  class Movie
9  {
10 public:
11     string title;
12     int rating;
13
14     Movie(string title, int rating)
15     {
16         this->title = title;
17         this->rating = rating;
18     }
19 };
20
21 void saveMovies(const vector<Movie>& movies)
22 {
23     ofstream file("movies.txt");
24
25     if (!file.is_open())
26     {
27         cout << "Failed to open file." << endl;
28         return;
29     }
30
31     for (const Movie& movie : movies)
32     {
33         file << movie.title << " " << movie.rating << endl;
34     }
35
36     file.close();
37
38     cout << "Movies saved successfully." << endl;

```

```

39 }
40
41 int main()
42 {
43     vector<Movie> movies;
44
45     movies.push_back(Movie("Interstellar", 9));
46     movies.push_back(Movie("Inception", 8));
47     movies.push_back(Movie("Tenet", 7));
48
49     saveMovies(movies);
50
51     return 0;
52 }

```

7. File Load: ifstream

ifstream을 사용하면 파일에 저장된 데이터를 다시 프로그램으로 읽어올 수 있다. Day 14에서는 파일에서 읽은 데이터를 이용해 객체를 생성하고 vector에 추가한다.

7.1. 파일에서 객체 복원

```

1 string title;
2 int price;
3
4 while (file >> title >> price)
5 {
6     books.push_back(Book(title, price));
7 }

```

while 조건에서 파일 입력이 성공하는 동안 반복하고, 읽은 데이터로 새로운 Book 객체를 만들어 원본 vector에 추가한다. 이 함수는 vector를 수정하므로 const를 붙이지 않은 참조를 사용한다.

7.2. 최종 코드

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <fstream>
5
6 using namespace std;
7
8 class Book
9 {
10 public:
11     string title;
12     int price;
13
14     Book(string title, int price)
15     {
16         this->title = title;
17         this->price = price;
18     }
19
20     void showInfo() const
21     {
22         cout << "Title: " << title << endl;

```

```

23     cout << "Price: " << price << endl;
24 }
25 };
26
27 void loadBooks(vector<Book>& books)
28 {
29     ifstream file("books.txt");
30
31     if (!file.is_open())
32     {
33         cout << "Failed to open file." << endl;
34         return;
35     }
36
37     string title;
38     int price;
39
40     while (file >> title >> price)
41     {
42         books.push_back(Book(title, price));
43     }
44
45     file.close();
46
47     cout << "Books loaded successfully." << endl;
48 }
49
50 void showBooks(const vector<Book>& books)
51 {
52     for (const Book& book : books)
53     {
54         book.showInfo();
55         cout << endl;
56     }
57 }
58
59 int main()
60 {
61     vector<Book> books;
62
63     loadBooks(books);
64
65     cout << endl;
66
67     showBooks(books);
68
69     return 0;
70 }

```

7.3. 실습

실습

movies.txt의 영화 데이터를 ifstream으로 읽고, 각 줄의 제목과 평점으로 Movie 객체를 만들어 vector<Movie>에 추가한다. 불러온 뒤 모든 영화를 출력한다.

```

1 #include <iostream>
2 #include <string>

```

```
3 #include <vector>
4 #include <fstream>
5
6 using namespace std;
7
8 class Movie
9 {
10 public:
11     string title;
12     int rating;
13
14     Movie(string title, int rating)
15     {
16         this->title = title;
17         this->rating = rating;
18     }
19
20     void showInfo() const
21     {
22         cout << title << " " << rating << endl;
23     }
24 };
25
26 void loadMovies(vector<Movie>& movies)
27 {
28     ifstream file("movies.txt");
29
30     if (!file.is_open())
31     {
32         cout << "Failed to open file." << endl;
33         return;
34     }
35
36     string title;
37     int rating;
38
39     while (file >> title >> rating)
40     {
41         movies.push_back(Movie(title, rating));
42     }
43
44     file.close();
45
46     cout << "Movies loaded successfully." << endl;
47 }
48
49 void showMovies(const vector<Movie>& movies)
50 {
51     for (const Movie& movie : movies)
52     {
53         movie.showInfo();
54     }
55 }
56
57 int main()
58 {
59     vector<Movie> movies;
60
61     loadMovies(movies);
62 }
```



```

63     showMovies(movies);
64
65     return 0;
66 }

```

8. Exception Handling 통합

파일 작업에서는 파일이 존재하지 않거나 열리지 않는 상황이 발생할 수 있다. 기존에는 if와 return으로 처리했지만, 이번에는 throw로 예외를 발생시키고 catch에서 처리한다.

8.1. 파일 오류와 runtime_error

```

1  if (!file.is_open())
2  {
3      throw runtime_error("Failed to open file.");
4  }

```

예외가 발생하면 현재 try 블록의 나머지 코드는 실행되지 않고 대응되는 catch로 제어가 이동한다.

```

1  catch (const exception& e)
2  {
3      cout << "Error: " << e.what() << endl;
4  }

```

8.2. 최종 코드

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <fstream>
5  #include <stdexcept>
6
7  using namespace std;
8
9  class Book
10 {
11 public:
12     string title;
13     int price;
14
15     Book(string title, int price)
16     {
17         this->title = title;
18         this->price = price;
19     }
20
21     void showInfo() const
22     {
23         cout << "Title: " << title << endl;
24         cout << "Price: " << price << endl;
25     }
26 };
27
28 void loadBooks(vector<Book>& books)
29 {
30     try

```

```

31 {
32     ifstream file("books.txt");
33
34     if (!file.is_open())
35     {
36         throw runtime_error("Failed to open file.");
37     }
38
39     string title;
40     int price;
41
42     while (file >> title >> price)
43     {
44         books.push_back(Book(title, price));
45     }
46
47     file.close();
48
49     cout << "Books loaded successfully." << endl;
50 }
51 catch (const exception& e)
52 {
53     cout << "Error: " << e.what() << endl;
54 }
55 }
56
57 int main()
58 {
59     vector<Book> books;
60
61     loadBooks(books);
62
63     return 0;
64 }

```

8.3. 실습

실습

movies.txt를 읽는 코드를 try / throw / catch 구조로 작성한다. 파일이 열리지 않으면 runtime_error를 발생시키고, catch (const exception& e)에서 오류 메시지를 출력한다.

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <fstream>
5 #include <stdexcept>
6
7 using namespace std;
8
9 class Movie
10 {
11 public:
12     string title;
13     int rating;
14
15     Movie(string title, int rating)
16     {

```

```
17     this->title = title;
18     this->rating = rating;
19 }
20
21 void showInfo() const
22 {
23     cout << title << " " << rating << endl;
24 }
25 };
26
27 void loadMovies(vector<Movie>& movies)
28 {
29     try
30     {
31         ifstream file("movies.txt");
32
33         if (!file.is_open())
34         {
35             throw runtime_error("Failed to open movies.txt");
36         }
37
38         string title;
39         int rating;
40
41         while (file >> title >> rating)
42         {
43             movies.push_back(Movie(title, rating));
44         }
45
46         file.close();
47
48         cout << "Movies loaded successfully." << endl;
49     }
50     catch (const exception& e)
51     {
52         cout << "Error: " << e.what() << endl;
53     }
54 }
55
56 int main()
57 {
58     vector<Movie> movies;
59
60     loadMovies(movies);
61
62     return 0;
63 }
```

9. 개념 연결

Day 14에서는 이전에 배운 기능들을 하나의 데이터 관리 흐름으로 연결한다.

이전 학습 내용	Day 14에서의 연결
함수	메뉴와 각 기능을 역할별 함수로 분리
참조	원본 <code>vector</code> 를 함수에서 직접 수정
<code>const</code>	읽기 전용 함수와 객체에 적용
클래스	<code>Book</code> , <code>Movie</code> 데이터 구조 정의
STL <code>vector</code>	여러 객체를 동적으로 관리
Lambda	검색 조건을 짧은 함수 형태로 작성
<code>find_if</code>	객체 컨테이너에서 조건 검색
File I/O	객체 데이터를 저장하고 다시 복원
Exception	파일 오류를 <code>throw</code> / <code>catch</code> 로 처리

핵심 포인트

데이터 관리 프로그램의 기본 흐름은 입력을 받아 객체를 만들고, 객체를 `vector`에 저장한 뒤 검색하거나 출력하고, 필요하면 파일에 저장하거나 파일에서 다시 불러오는 구조로 정리할 수 있다. 예외 처리는 이 과정에서 발생하는 오류를 정상 로직과 분리하여 처리한다.

10. 종합 정리

핵심 포인트

- `main()`은 프로그램의 전체 흐름을 관리하고 세부 기능은 함수로 분리할 수 있다.
- `vector<ClassName>` 형태로 여러 객체를 하나의 컨테이너에서 관리할 수 있다.
- 원본 `vector`를 수정해야 할 때는 참조 전달 `&`를 사용한다.
- 읽기만 하는 함수에서는 `const vector<T>&`를 사용하면 복사를 피하면서 수정을 막을 수 있다.
- `find_if`와 `lambda`를 조합하면 객체의 멤버 값을 기준으로 검색할 수 있다.
- 검색 실패 여부는 반환된 `iterator`가 `container.end()`와 같은지 확인한다.
- `ofstream`으로 `vector`의 객체 데이터를 파일에 저장할 수 있다.
- `ifstream`과 `while (file >> ...)`을 사용하면 파일 데이터를 반복해서 읽을 수 있다.
- 파일에서 읽은 값으로 객체를 생성한 뒤 `push_back()`하면 컨테이너를 복원할 수 있다.
- `throw runtime_error(...)`로 실행 중 오류를 예외로 전달할 수 있다.
- `catch (const exception& e)`와 `e.what()`으로 예외 메시지를 처리할 수 있다.
- Day 14의 목적은 개별 문법 암기가 아니라 Day 1-13의 개념을 실제 프로그램 흐름 안에서 연결하는 것이다.

11. 실습 파일 구성

실습

Day 14 파일은 다음과 같이 관리한다.

- 01_program_structure.cpp
- 02_class_and_vector.cpp
- 03_search_and_algorithm.cpp
- 04_file_save.cpp
- 05_file_load.cpp
- 06_exception_handling.cpp
- practice/p01_movie_manager.cpp
- practice/p02_movie_collection.cpp
- practice/p03_movie_search.cpp
- practice/p04_movie_save.cpp
- practice/p05_movie_load.cpp
- practice/p06_movie_exception.cpp
- practice/movies.txt